```
import asyncio
import logging
import random
import uuid
from datetime import datetime, date
from aiogram import Bot, Dispatcher, F
from aiogram.types import Message, CallbackQuery
from aiogram.filters import CommandStart, Command
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup
from aiogram.fsm.storage.memory import MemoryStorage
from aiogram.utils.keyboard import InlineKeyboardBuilder
from aiogram.exceptions import TelegramBadRequest

import config
import database as db
from questions import QUESTION_BANK, SUBJECTS, BRANCHES

logging.basicConfig(level=logging.INFO)
bot = Bot(token=config.BOT_TOKEN)
storage = MemoryStorage()
dp = Dispatcher(storage=storage)

MAX_PLAYERS = 25

SUBJECT_MAX_GRADES = {
    "islamic": 50, "arabic": 100, "english": 100, "french": 100,
    "math": 100, "physics": 100, "chemistry": 100, "biology": 100,
    "history": 100, "economics": 100, "geography": 100,
}

# ────────────────────────────────────────────
# FSM
# ────────────────────────────────────────────
class QuizStates(StatesGroup):
    choosing_subject = State()
    in_quiz          = State()
    study_mode       = State()
    challenge_wait   = State()

# ────────────────────────────────────────────
# Helpers
# ────────────────────────────────────────────
TITLES = [
```

```
    (0,    "🌱", "مبتدئ"),
    (100,  "📚", "متعلم"),
    (300,  "🎯", "متقدم"),
    (600,  "⭐", "محترف"),
    (1000, "🏆", "خبير"),
    (2000, "👑", "أسطورة"),
]

CERTIFICATES = {
    5: "🥉 شهادة المثابر البرونزية",
    15: "🥈 شهادة المثابر الفضية",
    30: "🥇 شهادة المثابر الذهبية",
    50: "💎 شهادة المثابر الماسية",
    100: "👑 شهادة أسطورة المثابر",
}

def get_title(xp):
    t = TITLES[0]
    for threshold, icon, label in TITLES:
        if xp >= threshold:
            t = (threshold, icon, label)
    return t

def get_certificate(games):
    cert = None
    for g, label in CERTIFICATES.items():
        if games >= g:
            cert = label
    return cert

def timer_bar(t, total=15):
    filled = int((t / total) * 10)
    char = "🟩" if t > 8 else ("🟨" if t > 4 else "🟥")
    return char * filled + "⬜" * (10 - filled) + f" {t}ث"

def progress_bar(cur, total):
    f = int((cur / total) * 10)
    return "▓" * f + "⬜" * (10 - f)

def subj_info(key):
    if key == "all":
        return "كل المواد", "🎲"
    s = SUBJECTS.get(key, {})
    return s.get("name", ""), s.get("icon", "📚")

def pick_questions(key, n=10):
    if key == "all":
```

```
        pool = [{**q, "_subject": k} for k, qs in QUESTION_BANK.items() for q in
    else:
        pool = QUESTION_BANK.get(key, [])
    selected = random.sample(pool, min(n, len(pool)))
    # خلط ترتيب الخيارات عشوائياً لكل سؤال
    return [shuffle_options(q) for q in selected]

def shuffle_options(q: dict) -> dict:
    """يخلط ترتيب الخيارات ويحدّث رقم الإجابة الصحيحة تلقائياً"""
    options = list(q["options"])
    correct_text = options[q["answer"]]  # نحفظ نص الإجابة الصحيحة
    # نخلط الخيارات
    indices = list(range(len(options)))
    random.shuffle(indices)
    new_options = [options[i] for i in indices]
    new_answer = new_options.index(correct_text)  # نحدد موضع الإجابة الصحيحة الجديد
    return {**q, "options": new_options, "answer": new_answer}

# ────────────────────────────────────────────
# Grade Prediction
# ────────────────────────────────────────────
def calc_grade_prediction(subject_stats: dict, branch: str = None) -> str:
    subjects_to_show = list(SUBJECTS.keys())
    if branch and branch in BRANCHES:
        subjects_to_show = BRANCHES[branch]["subjects"]

    lines = []
    total_weighted = 0
    total_max = 0

    for key in subjects_to_show:
        subj = SUBJECTS[key]
        max_grade = SUBJECT_MAX_GRADES.get(key, 100)
        s = subject_stats.get(key)

        if s and s["games"] > 0:
            acc = s["correct"] / (s["games"] * 10)
            predicted = round(acc * max_grade)
            pct = acc * 100
            status = ("🟢 ممتاز" if pct >= 90 else "🔵 جيد جداً" if pct >= 75 else
                      "🟡 جيد" if pct >= 60 else "🟠 مقبول" if pct >= 50 else "🔴
            bar = "▓" * int(pct // 10) + "░" * (10 - int(pct // 10))
            lines.append(
                f"{subj['icon']} *{subj['name']}*\n"
                f"  `{bar}` {int(pct)}%\n"
                f"  📝 التوقع: *{predicted}/{max_grade}* — {status}\n"
            )
```

```
            total_weighted += predicted
            total_max += max_grade
        else:
            lines.append(f"{subj['icon']} *{subj['name']}*\n ⚪ لم تراجع بعد\n")
            total_max += max_grade

    overall = round((total_weighted / total_max) * 100) if total_max > 0 else 0
    overall_label = (
        "🏆 ممتاز — أنت في المسار الصحيح!" if overall >= 85 else
        "⭐ جيد جداً — استمر بالمراجعة!"   if overall >= 70 else
        "💪 جيد — زد من جولاتك!"           if overall >= 55 else
        "📚 تحتاج مراجعة أكثر"             if overall > 0  else
        "العب جولات أولاً لترى توقعاتك!"
    )

    branch_name = BRANCHES[branch]["name"] if branch and branch in BRANCHES else
    return (
        f"🎯 *توقعات درجاتي — {branch_name}*\n"
        f"━━━━━━━━━━━━━━━━\n\n"
        + "\n".join(lines) +
        f"\n━━━━━━━━━━━━━━━━\n"
        f"📊 المعدل التقديري: *{overall}%*\n"
        f"{overall_label}\n\n"
        f"_⚠️ توقعات تقريبية بناءً على أدائك في البوت_"
    )

# ────────────────────────────────────────────
# Keyboards
# ────────────────────────────────────────────
def main_menu_kb():
    b = InlineKeyboardBuilder()
    b.button(text="📚 اختر المادة",        callback_data="subjects_menu")
    b.button(text="🎲 كل المواد",          callback_data="subject:all")
    b.button(text="📖 وضع الدراسة",        callback_data="study_mode_menu")
    b.button(text="⚔️ تحدي صديق",         callback_data="challenge_menu")
    b.button(text="👥 جولة مجموعة",        callback_data="group_menu")
    b.button(text="🏆 المتصدرون",          callback_data="leaderboard")
    b.button(text="📊 إحصائياتي",          callback_data="my_stats")
    b.button(text="🎯 توقعات درجاتي",      callback_data="grade_prediction_menu")
    b.button(text="📅 تذكير يومي",         callback_data="reminder_menu")
    b.button(text="📈 تقرير الأسبوعي",    callback_data="weekly_report")
    b.button(text="⚙️ الإعدادات",          callback_data="settings")
    b.adjust(2, 2, 2, 2, 2, 1)
    return b.as_markup()

def branch_select_kb(prefix="branch"):
    b = InlineKeyboardBuilder()
```

```
    for key, branch in BRANCHES.items():
        b.button(text=f"{branch['icon']} {branch['name']}", callback_data=f"{pref
    b.button(text="🌐 كل الفروع", callback_data=f"{prefix}:all")
    b.button(text="← رجوع", callback_data="back_home")
    b.adjust(1)
    return b.as_markup()

def subjects_for_branch_kb(branch_key: str, prefix="subject"):
    b = InlineKeyboardBuilder()
    if branch_key == "all":
        subjects = list(SUBJECTS.keys())
    else:
        subjects = BRANCHES[branch_key]["subjects"]
    for key in subjects:
        if key in ["arabic_grammar", "arabic_literature"]:
            continue
        subj = SUBJECTS[key]
        b.button(text=f"{subj['icon']} {subj['name']}", callback_data=f"{prefix}:
    b.button(text="🎲 كل مواد الفرع", callback_data=f"{prefix}:branch_{branch_key
    b.button(text="← رجوع", callback_data="subjects_menu")
    b.adjust(2)
    return b.as_markup()

def back_home_kb():
    b = InlineKeyboardBuilder()
    b.button(text="← القائمة الرئيسية", callback_data="back_home")
    return b.as_markup()

def options_kb(options, q_index, prefix="answer"):
    b = InlineKeyboardBuilder()
    labels = ["د" ,"ج" ,"ب" ,"أ"]
    for i, opt in enumerate(options):
        b.button(text=f"{labels[i]}) {opt}", callback_data=f"{prefix}:{q_index}:{
    b.adjust(1)
    return b.as_markup()

# ────────────────────────────────────────────
# Welcome
# ────────────────────────────────────────────
def welcome_text(user_id, first_name):
    stats = db.get_user_stats(user_id)
    _, icon, label = get_title(stats["xp"])
    cert = get_certificate(stats["total_games"])
    cert_line = f"🎖️ {cert}\n" if cert else ""
    return (
        f"👋 أهلاً {first_name}!\n\n"
        f"🎓 *المثابر الوزاري*\n"
```

```
        f"━━━━━━━━━━━━━━━━━\n"
        f"{cert_line}"
        f"{icon} لقبك: *{label}*\n"
        f"⚡ XP: *{stats['xp']}*\n"
        f"🔥 السلسلة: *{stats['streak']}* يوم\n"
        f"🎮 جولات: *{stats['total_games']}*\n\n"
        f"اختر ما تريد 👇"
    )

# ────────────────────────────────────────────
# /start
# ────────────────────────────────────────────
@dp.message(CommandStart())
async def cmd_start(message: Message, state: FSMContext):
    await state.clear()
    user = message.from_user
    db.register_user(user.id, user.full_name)

    args = message.text.split()
    if len(args) > 1 and args[1].startswith("session_"):
        await process_join(user, args[1][8:], message)
        return

    pending = db.get_pending_challenge(user.id)
    if pending:
        sn, si = subj_info(pending["subject"])
        b = InlineKeyboardBuilder()
        b.button(text="✅ قبول", callback_data=f"accept_challenge:{pending['id']}
        b.button(text="❌ رفض",  callback_data="back_home")
        b.adjust(2)
        await message.answer(
            f"⚔️ *تحدي جديد!*\n{si} *{sn}*\n هل تقبل؟",
            parse_mode="Markdown", reply_markup=b.as_markup()
        )
        return

    await message.answer(welcome_text(user.id, user.first_name),
                         parse_mode="Markdown", reply_markup=main_menu_kb())
    await state.set_state(QuizStates.choosing_subject)

@dp.callback_query(F.data == "back_home")
async def back_home(callback: CallbackQuery, state: FSMContext):
    await state.clear()
    user = callback.from_user
    try:
        await callback.message.edit_text(
            welcome_text(user.id, user.first_name),
```

```
            parse_mode="Markdown", reply_markup=main_menu_kb()
        )
    except TelegramBadRequest:
        await callback.message.answer(
            welcome_text(user.id, user.first_name),
            parse_mode="Markdown", reply_markup=main_menu_kb()
        )
    await state.set_state(QuizStates.choosing_subject)
    await callback.answer()

# ────────────────────────────────────────────
# Subjects Menu — Branch Selection
# ────────────────────────────────────────────
@dp.callback_query(F.data == "subjects_menu")
async def subjects_menu(callback: CallbackQuery):
    await callback.message.edit_text(
        "📚 *اختر المادة*\n\nأولاً اختر فرعك الدراسي:",
        parse_mode="Markdown",
        reply_markup=branch_select_kb("pick_branch")
    )
    await callback.answer()

@dp.callback_query(F.data == "study_mode_menu")
async async def study_mode_menu(callback: CallbackQuery):
    await callback.message.edit_text(
        "📖 *وضع الدراسة*\n\nاختر فرعك الدراسي:",
        parse_mode="Markdown",
        reply_markup=branch_select_kb("study_branch")
    )
    await callback.answer()

@dp.callback_query(F.data.startswith("pick_branch:"))
async def pick_branch(callback: CallbackQuery):
    branch_key = callback.data.split(":")[1]
    branch_name = BRANCHES[branch_key]["name"] if branch_key != "all" else "لفروع
    branch_icon = BRANCHES[branch_key]["icon"] if branch_key != "all" else "🌐"
    await callback.message.edit_text(
        f"{branch_icon} *{branch_name}*\n\nاختر المادة:",
        parse_mode="Markdown",
        reply_markup=subjects_for_branch_kb(branch_key, "subject")
    )
    await callback.answer()

@dp.callback_query(F.data.startswith("study_branch:"))
async def study_branch(callback: CallbackQuery):
    branch_key = callback.data.split(":")[1]
    branch_name = BRANCHES[branch_key]["name"] if branch_key != "all" else "لفروع
```

```
    branch_icon = BRANCHES[branch_key]["icon"] if branch_key != "all" else "🌐"
    await callback.message.edit_text(
        f"{branch_icon} *{branch_name}*\n\nاختر المادة:",
        parse_mode="Markdown",
        reply_markup=subjects_for_branch_kb(branch_key, "study")
    )
    await callback.answer()

# ═══════════════════════════════════════════════
# SOLO QUIZ
# ═══════════════════════════════════════════════
@dp.callback_query(F.data.startswith("subject:"))
async def choose_subject(callback: CallbackQuery, state: FSMContext):
    key = callback.data.split(":")[1]

    if key == "arabic":
        b = InlineKeyboardBuilder()
        b.button(text="📖 قواعد", callback_data="subject:arabic_grammar")
        b.button(text="📚 أدب", callback_data="subject:arabic_literature")
        b.button(text="← رجوع", callback_data="subjects_menu")
        b.adjust(2, 1)
        await callback.message.edit_text("📝 *العربي*\n\nماذا تريد أن تختبر؟", pa
        await callback.answer()
        return

    # Handle branch-wide selection
    if key.startswith("branch_"):
        branch_key = key.replace("branch_", "")
        if branch_key == "all":
            qs = pick_questions("all")
        else:
            branch_subjects = BRANCHES[branch_key]["subjects"]
            pool = [{**q, "_subject": k} for k in branch_subjects for q in QUESTI
            qs = random.sample(pool, min(10, len(pool)))
        display_key = key
    else:
        qs = pick_questions(key)
        display_key = key

    sn, si = subj_info(key if not key.startswith("branch_") else "all")
    await state.update_data(subject=display_key, questions=qs, current=0,
                            score=0, answers=[], mode="quiz", quiz_msg_id=None)
    sent = await callback.message.answer(
        f"{si} *{sn}*\n\nجاري تحضير الأسئلة... ⏳", parse_mode="Markdown"
    )
    await state.update_data(quiz_msg_id=sent.message_id)
    await state.set_state(QuizStates.in_quiz)
```

```
    await callback.answer()
    await asyncio.sleep(1)
    await run_solo_q(callback.message.chat.id, state)

async def build_q_text(q, cur, total, t, key):
    sn, si = subj_info(key if not key.startswith("branch_") else "all")
    return (
        f"{si} *{sn}* | {cur+1}/{total}\n"
        f"`{progress_bar(cur, total)}`\n\n"
        f"❓ *{q['q']}*\n\n"
        f"⏱️ {timer_bar(t)}"
    )

async def show_answer_and_next(chat_id, state: FSMContext, q, cur, total, key, ms
    """عرض الإجابة الصحيحة ثم الانتقال للسؤال التالي"""
    sn, si = subj_info(key if not key.startswith("branch_") else "all")
    labels = ["د" ,"ج" ,"ب" ,"أ"]
    correct_label = f"{labels[q['answer']]}) {q['options'][q['answer']]}"

    if timeout:
        header = "⏰ *انتهى الوقت!*"
    else:
        header = "✅ *انتهى السؤال!*"

    try:
        await bot.edit_message_text(
            chat_id=chat_id, message_id=msg_id,
            text=(
                f"{si} *{sn}* | {cur+1}/{total}\n"
                f"`{progress_bar(cur+1, total)}`\n\n"
                f"❓ *{q['q']}*\n\n"
                f"{header}\n"
                f"✅ الإجابة الصحيحة: *{correct_label}*\n\n"
                f"التالي خلال ثانيتين..."
            ),
            parse_mode="Markdown"
        )
    except TelegramBadRequest:
        pass
    await asyncio.sleep(2)
    await run_solo_q(chat_id, state)

async def run_solo_q(chat_id, state: FSMContext):
    data = await state.get_data()
    qs, cur, key = data["questions"], data["current"], data["subject"]
    msg_id = data["quiz_msg_id"]
    if cur >= len(qs):
```

```
        await solo_finish(chat_id, state)
        return
    q = qs[cur]
    total = len(qs)
    await state.update_data(timer_running=True)
    text = await build_q_text(q, cur, total, 15, key)
    try:
        await bot.edit_message_text(chat_id=chat_id, message_id=msg_id,
                                    text=text, parse_mode="Markdown",
                                    reply_markup=options_kb(q["options"], cur))
    except TelegramBadRequest:
        pass
    for t in range(14, -1, -1):
        await asyncio.sleep(1)
        data = await state.get_data()
        if not data.get("timer_running"):
            return  # تم الإجابة — handle_solo_answer يتولى الأمر
        text = await build_q_text(q, cur, total, t, key)
        try:
            await bot.edit_message_text(chat_id=chat_id, message_id=msg_id,
                                        text=text, parse_mode="Markdown",
                                        reply_markup=options_kb(q["options"], cur
        except TelegramBadRequest:
            pass

    # انتهى الوقت بدون إجابة
    data = await state.get_data()
    if not data.get("timer_running"):
        return
    await state.update_data(timer_running=False)
    answers = data.get("answers", [])
    answers.append({"q": q["q"], "options": q["options"], "correct": q["answer"],
    await state.update_data(current=cur+1, answers=answers)
    await show_answer_and_next(chat_id, state, q, cur, total, key, msg_id, timeou

@dp.callback_query(F.data.startswith("answer:"), QuizStates.in_quiz)
async def handle_solo_answer(callback: CallbackQuery, state: FSMContext):
    parts = callback.data.split(":")
    q_idx, chosen = int(parts[1]), int(parts[2])
    data = await state.get_data()
    if data.get("current") != q_idx or not data.get("timer_running"):
        await callback.answer("⚡ تأخرت!")
        return

    # أوقف المؤقت فوراً
    await state.update_data(timer_running=False)
```

```
    qs, answers, score = data["questions"], data.get("answers", []), data["score"
    q, key, msg_id = qs[q_idx], data["subject"], data["quiz_msg_id"]
    chat_id = callback.message.chat.id
    correct = chosen == q["answer"]
    if correct:
        score += 1
    answers.append({"q": q["q"], "options": q["options"], "correct": q["answer"],
    await state.update_data(current=q_idx+1, score=score, answers=answers)

    sn, si = subj_info(key if not key.startswith("branch_") else "all")
    labels = ["د" ,"ج" ,"ب" ,"أ"]
    correct_label = f"{labels[q['answer']]}) {q['options'][q['answer']]}"
    total = len(qs)

    # عرض النتيجة + الإجابة الصحيحة فوراً
    if correct:
        result_text = f"✅ *10+ XP\n✅ إجابة صحيحة!* الإجابة: *{correct_label}*"
    else:
        chosen_label = f"{labels[chosen]}) {q['options'][chosen]}"
        result_text = f"❌ *إجابة خاطئة!*\nإجابتك: {chosen_label}\n✅ الصحيحة: *{

    try:
        await bot.edit_message_text(
            chat_id=chat_id, message_id=msg_id,
            text=(
                f"{si} *{sn}* | {q_idx+1}/{total}\n"
                f"`{progress_bar(q_idx+1, total)}`\n\n"
                f"❓ *{q['q']}*\n\n"
                f"{result_text}\n\n"
                f"التالي خلال ثانيتين..."
            ),
            parse_mode="Markdown"
        )
    except TelegramBadRequest:
        pass

    await callback.answer("✅ صحيح!" if correct else "❌ خطأ!")
    await asyncio.sleep(2)
    await run_solo_q(chat_id, state)

async def solo_finish(chat_id, state: FSMContext):
    data = await state.get_data()
    answers, key, mode = data["answers"], data["subject"], data.get("mode", "quiz
    msg_id = data["quiz_msg_id"]
    score = sum(1 for a in answers if a["chosen"] == a["correct"])
    total = len(answers)
    pct = int((score/total)*100) if total else 0
```

```
xp = score * 10
grade = ("🌟 ممتاز" if pct>=90 else "⭐ جيد جداً" if pct>=70 else
         "👍 جيد" if pct>=50 else "📚 تحتاج مراجعة")
if mode == "quiz":
    actual = key if not key.startswith("branch_") and key != "all" else "mixe
    db.update_stats(chat_id, xp, score, total, actual)
stats = db.get_user_stats(chat_id)
_, icon, label = get_title(stats["xp"])
cert = get_certificate(stats["total_games"])
cert_line = f"🎖️ حصلت على: *{cert}*\n" if cert and mode == "quiz" else ""

cid = data.get("challenge_id")
is_ch = data.get("is_challenger", False)
if cid:
    db.update_challenge_score(cid, is_ch, score)
    challenge = db.get_challenge(cid)
    if challenge and challenge["status"] == "finished":
        c_s, o_s = challenge["challenger_score"], challenge["opponent_score"]
        res = "🤝 تعادل!" if c_s == o_s else ("🏆 فزت!" if (is_ch and c_s > c
        try:
            await bot.edit_message_text(chat_id=chat_id, message_id=msg_id,
                text=f"⚔️ *نتيجة التحدي*\n━━━━━━━━━━━━━━━━━━\nالمتحدي: {c_s}/10\
                parse_mode="Markdown", reply_markup=back_home_kb())
        except TelegramBadRequest:
            pass
        await state.clear()
        return

sn, si = subj_info(key if not key.startswith("branch_") else "all")
b = InlineKeyboardBuilder()
b.button(text="👁️ مراجعة الإجابات", callback_data=f"review:0:{key}")
b.button(text="🔄 جولة جديدة",      callback_data=f"subject:{key}")
b.button(text="📚 تغيير المادة",    callback_data="subjects_menu")
b.button(text="🎯 توقعات درجاتي",  callback_data="grade_prediction_menu")
b.adjust(1, 2, 1)
try:
    await bot.edit_message_text(chat_id=chat_id, message_id=msg_id,
        text=(f"🏁 *انتهت الجولة!*\n\n{si} *{sn}*\n━━━━━━━━━━━━━━━━━━\n"
              f"✅ صحيح: *{score}/{total}*\n📊 النسبة: *{pct}%*\n"
              f"⚡ XP: *+{xp}*\n━━━━━━━━━━━━━━━━━━\n"
              f"🎖️ التقييم: *{grade}*\n{cert_line}"
              f"{icon} لقبك: {label}\n🔥 السلسلة: {stats['streak']} يوم\n"
              f"⚡ إجمالي XP: {stats['xp']}"),
        parse_mode="Markdown", reply_markup=b.as_markup())
except TelegramBadRequest:
    pass
await state.clear()
```

```
    await state.set_state(QuizStates.choosing_subject)

# ═══════════════════════════════════════════════
# STUDY MODE
# ═══════════════════════════════════════════════
@dp.callback_query(F.data.startswith("study:"))
async def start_study(callback: CallbackQuery, state: FSMContext):
    key = callback.data.split(":")[1]

    if key == "arabic":
        b = InlineKeyboardBuilder()
        b.button(text="📖 قواعد", callback_data="study:arabic_grammar")
        b.button(text="📚 أدب", callback_data="study:arabic_literature")
        b.button(text="← رجوع", callback_data="study_mode_menu")
        b.adjust(2, 1)
        await callback.message.edit_text("📝 *العربي*\n\n؟ماذا تريد أن تدرس", par
        await callback.answer()
        return
    if key.startswith("branch_"):
        branch_key = key.replace("branch_", "")
        branch_subjects = BRANCHES[branch_key]["subjects"] if branch_key != "all"
        pool = [{**q, "_subject": k} for k in branch_subjects for q in QUESTION_B
        qs = random.sample(pool, min(10, len(pool)))
    else:
        qs = pick_questions(key)
    await state.update_data(subject=key, questions=qs, current=0,
                            score=0, answers=[], mode="study")
    await callback.message.edit_text("📖 *خذ وقتك — *وضع الدراسة 😊", parse_mode=
    await asyncio.sleep(1)
    await send_study_q(callback.message, state)
    await state.set_state(QuizStates.study_mode)
    await callback.answer()

async def send_study_q(message: Message, state: FSMContext):
    data = await state.get_data()
    qs, cur, key = data["questions"], data["current"], data["subject"]
    if cur >= len(qs):
        await solo_finish(message.chat.id, state)
        return
    q = qs[cur]
    total = len(qs)
    sn, si = subj_info(key if not key.startswith("branch_") else "all")
    b = InlineKeyboardBuilder()
    for i, opt in enumerate(q["options"]):
        labels = ["د" ,"ج" ,"ب" ,"أ"]
        b.button(text=f"{labels[i]}) {opt}", callback_data=f"study_ans:{cur}:{i}"
    b.adjust(1)
```

```
    await message.answer(
        f"{si} *{sn}* | {cur+1}/{total}\n`{progress_bar(cur, total)}`\n\n📖 *{q['
        parse_mode="Markdown", reply_markup=b.as_markup()
    )
    await state.update_data(quiz_msg_id=None)

@dp.callback_query(F.data.startswith("study_ans:"), QuizStates.study_mode)
async def handle_study_ans(callback: CallbackQuery, state: FSMContext):
    parts = callback.data.split(":")
    q_idx, chosen = int(parts[1]), int(parts[2])
    data = await state.get_data()
    if data.get("current") != q_idx:
        await callback.answer()
        return
    qs, score, answers = data["questions"], data["score"], data.get("answers", []
    q = qs[q_idx]
    correct = chosen == q["answer"]
    if correct:
        score += 1
    answers.append({"q": q["q"], "options": q["options"], "correct": q["answer"],
    await state.update_data(current=q_idx+1, score=score, answers=answers,
                            quiz_msg_id=callback.message.message_id)
    feedback = "✅ *صحيح!*" if correct else f"❌ *خطأ!*\n✅ الصحيحة: *{q['options
    b = InlineKeyboardBuilder()
    b.button(text="التالي ←", callback_data=f"study_next:{q_idx}")
    try:
        await callback.message.edit_text(f"*{q['q']}*\n\n{feedback}",
                                         parse_mode="Markdown", reply_markup=b.as
    except TelegramBadRequest:
        pass
    await callback.answer("✅" if correct else "❌")

@dp.callback_query(F.data.startswith("study_next:"))
async def study_next(callback: CallbackQuery, state: FSMContext):
    await send_study_q(callback.message, state)
    await callback.answer()

# ═══════════════════════════════════════════════
# GRADE PREDICTION
# ═══════════════════════════════════════════════
@dp.callback_query(F.data == "grade_prediction_menu")
async def grade_prediction_menu(callback: CallbackQuery):
    await callback.message.edit_text(
        "🎯 *توقعات درجاتي بالوزاري*\n\nاختر فرعك لعرض التوقعات:",
        parse_mode="Markdown",
        reply_markup=branch_select_kb("grade_branch")
    )
```

```
    await callback.answer()

@dp.callback_query(F.data.startswith("grade_branch:"))
async def grade_branch(callback: CallbackQuery):
    branch_key = callback.data.split(":")[1]
    subject_stats = db.get_subject_stats(callback.from_user.id)
    text = calc_grade_prediction(subject_stats, branch_key if branch_key != "all"

    b = InlineKeyboardBuilder()
    b.button(text="🔄 فرع آخر", callback_data="grade_prediction_menu")
    b.button(text="← رجوع",    callback_data="back_home")
    b.adjust(2)
    await callback.message.edit_text(text, parse_mode="Markdown", reply_markup=b.
    await callback.answer()

# ═══════════════════════════════════════════════
# GROUP QUIZ
# ═══════════════════════════════════════════════
group_sessions = {}

@dp.callback_query(F.data == "group_menu")
async def group_menu(callback: CallbackQuery):
    await callback.message.edit_text(
        "👥 *جولة المجموعة*\n\nاختر فرعك أولاً:",
        parse_mode="Markdown",
        reply_markup=branch_select_kb("group_branch")
    )
    await callback.answer()

@dp.callback_query(F.data.startswith("group_branch:"))
async def group_branch(callback: CallbackQuery):
    branch_key = callback.data.split(":")[1]
    branch_name = BRANCHES[branch_key]["name"] if branch_key != "all" else "لفروع
    await callback.message.edit_text(
        f"👥 *جولة المجموعة — {branch_name}*\n\nاختر المادة:",
        parse_mode="Markdown",
        reply_markup=subjects_for_branch_kb(branch_key, "group_create")
    )
    await callback.answer()

@dp.callback_query(F.data.startswith("group_create:"))
async def group_create(callback: CallbackQuery):
    key = callback.data.split(":")[1]

    if key == "arabic":
        b = InlineKeyboardBuilder()
        b.button(text="📖 قواعد", callback_data="group_create:arabic_grammar")
```

```
        b.button(text="📚 أدب", callback_data="group_create:arabic_literature")
        b.button(text="← رجوع", callback_data="group_menu")
        b.adjust(2, 1)
        await callback.message.edit_text("📝 *اختر نوع الجولة:*\n\nالعربي", parse
        await callback.answer()
        return
    user = callback.from_user
    sid = str(uuid.uuid4())[:8].upper()
    sn, si = subj_info(key if not key.startswith("branch_") else "all")
    bot_info = await bot.get_me()

    if key.startswith("branch_"):
        branch_key = key.replace("branch_", "")
        branch_subjects = BRANCHES[branch_key]["subjects"] if branch_key != "all"
        pool = [{**q, "_subject": k} for k in branch_subjects for q in QUESTION_B
        questions = random.sample(pool, min(10, len(pool)))
    else:
        questions = pick_questions(key)

    group_sessions[sid] = {
        "subject": key, "owner_id": user.id, "owner_name": user.full_name,
        "status": "waiting", "chat_id": None, "msg_id": None,
        "questions": questions, "current": 0,
        "players": {user.id: {"name": user.full_name, "score": 0}},
        "answered": {},
    }

    share_link = f"https://t.me/{bot_info.username}?start=session_{sid}"
    b = InlineKeyboardBuilder()
    b.button(text="🚀 ابدأ في هذه المحادثة", callback_data=f"start_here:{sid}")
    b.adjust(1)

    await callback.message.edit_text(
        f"✅ *تم إنشاء الجولة!*\n\n{si} *{sn}*\n🔑 الكود: `{sid}`\n\n"
        f"*شارك هذا الأمر بمجموعتك:*\n`/startquiz {sid}`\n\n"
        f"*أو الرابط المباشر:*\n{share_link}",
        parse_mode="Markdown", reply_markup=b.as_markup()
    )
    await callback.answer()

@dp.message(Command("startquiz"))
async def cmd_startquiz(message: Message):
    parts = message.text.split()
    if len(parts) == 1:
        b = InlineKeyboardBuilder()
        for bkey, branch in BRANCHES.items():
            b.button(text=f"{branch['icon']} {branch['name']}",
```

```
                    callback_data=f"quick_branch:{bkey}:{message.chat.id}")
        b.adjust(1)
        await message.answer("👥 *جولة مجموعة*\n\nاختر الفرع:", parse_mode="Markd
        return
    sid = parts[1].upper()
    if sid not in group_sessions:
        await message.answer("❌ كود الجولة غير صحيح!")
        return
    session = group_sessions[sid]
    if session["owner_id"] != message.from_user.id:
        await message.answer("❌ فقط صاحب الجولة يقدر يبدأ!")
        return
    session["chat_id"] = message.chat.id
    await launch_group_quiz(message.chat.id, sid)

@dp.callback_query(F.data.startswith("quick_branch:"))
async def quick_branch(callback: CallbackQuery):
    parts = callback.data.split(":")
    branch_key = parts[1]
    chat_id = int(parts[2])
    await callback.message.edit_text(
        "اختر المادة:",
        reply_markup=subjects_for_branch_kb(branch_key, f"quick_quiz_{chat_id}")
    )
    await callback.answer()

@dp.callback_query(F.data.startswith("start_here:"))
async def start_here(callback: CallbackQuery):
    sid = callback.data.split(":")[1]
    if sid not in group_sessions:
        await callback.answer("❌ الجولة غير موجودة!", show_alert=True)
        return
    session = group_sessions[sid]
    if session["owner_id"] != callback.from_user.id:
        await callback.answer("❌ فقط صاحب الجولة!", show_alert=True)
        return
    session["chat_id"] = callback.message.chat.id
    await callback.answer("🚀 تبدأ الجولة!")
    await launch_group_quiz(callback.message.chat.id, sid)

async def process_join(user, sid, source):
    if sid not in group_sessions:
        txt = "❌ الجولة غير موجودة!"
        if isinstance(source, CallbackQuery):
            await source.answer(txt, show_alert=True)
        else:
            await source.answer(txt)
```

```
    return

session = group_sessions[sid]

if session["status"] != "waiting":
    txt = "❌ الجولة بدأت بالفعل!"
    if isinstance(source, CallbackQuery):
        await source.answer(txt, show_alert=True)
    else:
        await source.answer(txt)
    return

if len(session["players"]) >= MAX_PLAYERS:
    txt = "❌ الجولة ممتلئة!"
    if isinstance(source, CallbackQuery):
        await source.answer(txt, show_alert=True)
    else:
        await source.answer(txt)
    return

# ── تحقق إذا الشخص فعّل البوت ──────────────────
bot_info = await bot.get_me()
activate_link = f"https://t.me/{bot_info.username}?start=session_{sid}"

# نحاول نرسل له رسالة خاصة — إذا فشلت يعني ما فعّل البوت
bot_activated = True
try:
    await bot.send_message(
        user.id,
        f"⏳ *جاري الانضمام للجولة...*\n\nسيبدأ البوت بإرسال الأسئلة لك هنا.",
        parse_mode="Markdown"
    )
except Exception:
    bot_activated = False

if not bot_activated:
    # أخبر المجموعة إنه يحتاج يفعّل البوت
    group_chat_id = session.get("chat_id")

    if isinstance(source, CallbackQuery):
        await source.answer("⚠️ يجب تفعيل البوت أولاً!", show_alert=True)

    # رسالة في المجموعة مع زر تفعيل
    b = InlineKeyboardBuilder()
    b.button(
        text="🤖 فعّل البوت وانضم للجولة",
        url=activate_link
```

```
    )
    notification_text = (
        f"⚠️ {user.first_name} يحتاج تفعيل البوت أولاً للمشاركة!\n\n"
        f"👇 اضغط الزر أدناه لتفعيل البوت والانضمام للجولة"
    )

    try:
        if group_chat_id:
            await bot.send_message(
                group_chat_id,
                notification_text,
                parse_mode="Markdown",
                reply_markup=b.as_markup()
            )
        elif isinstance(source, CallbackQuery):
            await source.message.answer(
                notification_text,
                parse_mode="Markdown",
                reply_markup=b.as_markup()
            )
    except Exception:
        pass
    return

# ── انضم بنجاح ──────────────────────────────
session["players"][user.id] = {"name": user.full_name, "score": 0}
sn, si = subj_info(session["subject"] if not session["subject"].startswith("b

# رسالة خاصة تأكيد الانضمام
try:
    await bot.send_message(
        user.id,
        f"✅ *انضممت للجولة!*\n\n"
        f"{si} *{sn}*\n"
        f"👥 اللاعبون حالياً: {len(session['players'])}\n\n"
        f"⏳ انتظر حتى يبدأ صاحب الجولة...",
        parse_mode="Markdown"
    )
except Exception:
    pass

if isinstance(source, CallbackQuery):
    await source.answer(f"✅ انضممت! ({len(session['players'])} لاعب)")
elif isinstance(source, Message):
    await source.answer(
        f"✅ *انضممت للجولة!*\n{si} *{sn}*\nاللاعبون: {len(session['players'])
        parse_mode="Markdown"
```

```
        )

async def launch_group_quiz(chat_id: int, sid: str):
    session = group_sessions[sid]
    session["status"] = "active"
    session["chat_id"] = chat_id  # تأكد إن chat_id محفوظ
    qs = session["questions"]
    key = session["subject"]
    sn, si = subj_info(key if not key.startswith("branch_") else "all")
    total = len(qs)

    bot_info = await bot.get_me()
    activate_link = f"https://t.me/{bot_info.username}?start=session_{sid}"

    # رسالة البداية مع زر الانضمام
    b = InlineKeyboardBuilder()
    b.button(text="✋ انضم للجولة", url=activate_link)
    b.adjust(1)

    await bot.send_message(
        chat_id,
        f"🚀 *جولة المثابر الوزاري بدأت!*\n"
        f"{si} *{sn}* — {total} أسئلة\n"
        f"⏱️ 15 ثانية لكل سؤال\n\n"
        f"⚠️ *للمشاركة يجب تفعيل البوت أولاً*\n"
        f"اضغط الزر أدناه 👇\n\n"
        f"3️⃣2️⃣1️⃣ استعدوا...",
        parse_mode="Markdown",
        reply_markup=b.as_markup()
    )
    await asyncio.sleep(5)

    bot_info = await bot.get_me()
    activate_link_base = f"https://t.me/{bot_info.username}?start=session_{sid}"

    for q_idx, q in enumerate(qs):
        session["answered"] = {}
        session["current"]  = q_idx
        session["q_done"]   = False
        labels = ["أ", "ب", "ج", "د"]
        activate_link = activate_link_base

        def build_group_q(t, _q=q, _q_idx=q_idx):
            names = [p["name"] for uid, p in session["players"].items() if uid in
            ans_line = f"\n\n✅ أجاب: {', '.join(names)} ({len(names)}/{len(sessi
            return (f"{si} *{sn}* | {_q_idx+1}/{total}\n`{progress_bar(_q_idx, to
                    f"❓ *{_q['q']}*\n\n⏱️ {timer_bar(t)}{ans_line}")
```

```
def answer_kb(_q_idx=q_idx):
    b = InlineKeyboardBuilder()
    for i, opt in enumerate(q["options"]):
        b.button(text=f"{labels[i]}) {opt}", callback_data=f"gq:{sid}:{_q
    b.button(text="✋ انضم للجولة", url=activate_link)
    b.adjust(1)
    return b.as_markup()

sent = await bot.send_message(chat_id, build_group_q(15), parse_mode="Mar
session["msg_id"] = sent.message_id

# Countdown — ينتهي مبكراً إذا أجاب الكل
for t in range(14, -1, -1):
    await asyncio.sleep(1)

    # تحقق إذا أجاب الكل مبكراً
    if session.get("q_done"):
        break

    all_answered = len(session["answered"]) >= len(session["players"])
    if all_answered:
        session["q_done"] = True
        break

    try:
        await bot.edit_message_text(chat_id=chat_id, message_id=sent.mess
            text=build_group_q(t), parse_mode="Markdown",
            reply_markup=answer_kb() if t > 0 else None)
    except TelegramBadRequest:
        pass

# عرض الإجابة الصحيحة + النتيجة فوراً
correct_ans   = q["answer"]
correct_label = f"{labels[correct_ans]}) {q['options'][correct_ans]}"
correct_p = [p["name"] for uid, p in session["players"].items() if sessio
wrong_p   = [p["name"] for uid, p in session["players"].items() if uid in
no_ans_p  = [p["name"] for uid, p in session["players"].items() if uid no

# تحديث الرسالة بالإجابة الصحيحة
opts_text = "\n".join(
    f"{'✅' if i == correct_ans else '⬜'} {labels[i]}) {q['options'][i]}
    for i in range(len(q["options"]))
)
early = " (أجاب الكل مبكراً ⚡)" if session.get("q_done") else ""
try:
    await bot.edit_message_text(
```

```
                chat_id=chat_id, message_id=sent.message_id,
                text=(
                    f"{si} *{sn}* | {q_idx+1}/{total}\n"
                    f"`{progress_bar(q_idx+1, total)}`\n\n"
                    f"❓ *{q['q']}*\n\n{opts_text}\n\n"
                    f"✅ الإجابة الصحيحة: *{correct_label}*{early}"
                ),
                parse_mode="Markdown"
            )
        except TelegramBadRequest:
            pass

        # رسالة النتيجة
        res = f"📊 *نتيجة السؤال {q_idx+1}*\n✅ الصحيحة: *{correct_label}*\n\n"
        if correct_p: res += f"✅ أجاب صح ({len(correct_p)}): {', '.join(correct_
        if wrong_p:   res += f"❌ أخطأ ({len(wrong_p)}): {', '.join(wrong_p)}\n"
        if no_ans_p:  res += f"⏰ لم يجب ({len(no_ans_p)}): {', '.join(no_ans_p)}

        await bot.send_message(chat_id, res, parse_mode="Markdown")
        await asyncio.sleep(3)

    # Final results
    sorted_p = sorted(session["players"].items(), key=lambda x: x[1]["score"], re
    medals = ["🥇", "🥈", "🥉"]
    final = f"🏆 *النتائج النهائية*\n{si} *{sn}*\n━━━━━━━━━━━━━━━\n"
    for i, (uid, p) in enumerate(sorted_p):
        medal = medals[i] if i < 3 else f"{i+1}."
        pct = int((p["score"] / total) * 100)
        final += f"{medal} *{p['name']}* — {p['score']}/{total} ({pct}%)\n"
    final += "━━━━━━━━━━━━━━━\n🎉 تهانينا للمتصدرين!"

    b = InlineKeyboardBuilder()
    b.button(text="🔄 جولة جديدة", callback_data="group_menu")
    await bot.send_message(chat_id, final, parse_mode="Markdown", reply_markup=b.

    for uid, p in session["players"].items():
        if p["score"] > 0:
            db.update_stats(uid, p["score"]*10, p["score"], total, key if not key

    del group_sessions[sid]

@dp.callback_query(F.data.startswith("gq:"))
async def group_answer(callback: CallbackQuery):
    parts = callback.data.split(":")
    sid, q_idx, chosen = parts[1], int(parts[2]), int(parts[3])
    user = callback.from_user
```

```
if sid not in group_sessions:
    await callback.answer("❌ انتهت الجولة!")
    return

session = group_sessions[sid]
if user.id not in session["players"]:
    if len(session["players"]) >= MAX_PLAYERS:
        await callback.answer("❌ الجولة ممتلئة!", show_alert=True)
        return
    session["players"][user.id] = {"name": user.full_name, "score": 0}

if user.id in session["answered"]:
    await callback.answer("✋ أجبت بالفعل!")
    return

if session["current"] != q_idx:
    await callback.answer("⚡ تأخرت!")
    return

session["answered"][user.id] = chosen
q = session["questions"][q_idx]
correct = chosen == q["answer"]
if correct:
    session["players"][user.id]["score"] += 1

labels = ["أ", "ب", "ج", "د"]
await callback.answer("✅ صحيح! +1" if correct else f"❌ خطأ! الصحيحة: {label

# تحديث الرسالة بأسماء من أجاب
names = [p["name"] for uid, p in session["players"].items() if uid in session
sn, si = subj_info(session["subject"] if not session["subject"].startswith("b
total = len(session["questions"])

try:
    await callback.message.edit_text(
        f"{si} *{sn}* | {q_idx+1}/{total}\n`{progress_bar(q_idx, total)}`\n\n
        f"❓ *{q['q']}*\n\n"
        f"✅ أجاب: {', '.join(names)} ({len(names)}/{len(session['players'])}
        parse_mode="Markdown", reply_markup=callback.message.reply_markup
    )
except TelegramBadRequest:
    pass

# إذا أجاب الكل — أنهِ السؤال مبكراً
if len(session["answered"]) >= len(session["players"]):
    session["q_done"] = True
```

```
# ═══════════════════════════════════════════════
# CHALLENGE, LEADERBOARD, STATS, REVIEW, REMINDERS
# (بدون تغيير v4 نفس الكود من)
# ═══════════════════════════════════════════════
@dp.callback_query(F.data == "challenge_menu")
async def challenge_menu(callback: CallbackQuery):
    await callback.message.edit_text("⚔️ *تحدي صديق*\n\nاختر الفرع:",
                                     parse_mode="Markdown", reply_markup=branch_s
    await callback.answer()

@dp.callback_query(F.data.startswith("challenge_branch:"))
async def challenge_branch(callback: CallbackQuery):
    branch_key = callback.data.split(":")[1]
    await callback.message.edit_text("⚔️ اختر المادة:", parse_mode="Markdown",
                                     reply_markup=subjects_for_branch_kb(branch_k
    await callback.answer()

@dp.callback_query(F.data.startswith("challenge_subject:"))
async def challenge_select(callback: CallbackQuery, state: FSMContext):
    key = callback.data.split(":")[1]
    await state.update_data(challenge_subject=key)
    await callback.message.edit_text("⚔️ أرسل اسم صديقك في البوت:", parse_mode="M
    await state.set_state(QuizStates.challenge_wait)
    await callback.answer()

@dp.message(QuizStates.challenge_wait)
async def challenge_search(message: Message, state: FSMContext):
    results = db.get_user_by_username_search(message.text.strip())
    if not results:
        await message.answer("❌ ما وجدنا هذا المستخدم", reply_markup=back_home_k
        return
    data = await state.get_data()
    key = data.get("challenge_subject", "math")
    b = InlineKeyboardBuilder()
    for u in results:
        if u["user_id"] != message.from_user.id:
            b.button(text=f"⚔️ {u['name']}", callback_data=f"send_challenge:{u['u
    b.button(text="← رجوع", callback_data="back_home")
    b.adjust(1)
    await message.answer("اختر من تريد تتحداه:", reply_markup=b.as_markup())

@dp.callback_query(F.data.startswith("send_challenge:"))
async def send_challenge_cb(callback: CallbackQuery, state: FSMContext):
    parts = callback.data.split(":")
    opp, key = int(parts[1]), parts[2]
    cid = db.create_challenge(callback.from_user.id, opp, key)
    sn, si = subj_info(key if not key.startswith("branch_") else "all")
```

```
    try:
        b = InlineKeyboardBuilder()
        b.button(text="✅ قبول", callback_data=f"accept_challenge:{cid}")
        b.button(text="❌ رفض",  callback_data="back_home")
        b.adjust(2)
        await bot.send_message(opp, f"⚔️ *تحدي من {callback.from_user.full_name}!
                                parse_mode="Markdown", reply_markup=b.as_markup()
    except Exception:
        pass
    qs = pick_questions(key)
    sent = await callback.message.answer(f"✅ *تم إرسال التحدي!*\n{si} *{sn}*\nلآن
    await state.update_data(subject=key, questions=qs, current=0, score=0, answer
                            mode="challenge", challenge_id=cid, is_challenger=Tru
    await asyncio.sleep(2)
    await state.set_state(QuizStates.in_quiz)
    await run_solo_q(callback.message.chat.id, state)
    await callback.answer()

@dp.callback_query(F.data.startswith("accept_challenge:"))
async def accept_challenge_cb(callback: CallbackQuery, state: FSMContext):
    cid = int(callback.data.split(":")[1])
    challenge = db.get_challenge(cid)
    if not challenge:
        await callback.answer("انتهت صلاحية التحدي", show_alert=True)
        return
    key = challenge["subject"]
    sn, si = subj_info(key if not key.startswith("branch_") else "all")
    qs = pick_questions(key)
    sent = await callback.message.answer(f"⚔️ *قبلت التحدي!*\n{si} *{sn}*\nدأ الآن
    await state.update_data(subject=key, questions=qs, current=0, score=0, answer
                            mode="challenge", challenge_id=cid, is_challenger=Fal
    await asyncio.sleep(1.5)
    await state.set_state(QuizStates.in_quiz)
    await run_solo_q(callback.message.chat.id, state)
    await callback.answer()

@dp.callback_query(F.data == "leaderboard")
async def show_leaderboard(callback: CallbackQuery):
    top = db.get_leaderboard(10)
    medals = ["🥇", "🥈", "🥉"]
    if not top:
        await callback.message.edit_text("لا يوجد لاعبون بعد!", reply_markup=back_h
        return
    text = "🏆 *المتصدرون*\n━━━━━━━━━━━━━━━━━━\n"
    for i, row in enumerate(top):
        medal = medals[i] if i < 3 else f"{i+1}."
        _, icon, label = get_title(row["xp"])
```

```
        text += f"{medal} *{row['name']}* {icon}\n    ⚡ {row['xp']} XP  🔥 {row|
    await callback.message.edit_text(text, parse_mode="Markdown", reply_markup=ba
    await callback.answer()

@dp.callback_query(F.data == "my_stats")
async def show_my_stats(callback: CallbackQuery):
    uid = callback.from_user.id
    stats = db.get_user_stats(uid)
    subject_stats = db.get_subject_stats(uid)
    _, icon, label = get_title(stats["xp"])
    cert = get_certificate(stats["total_games"])
    text = (f"📊 *إحصائياتي*\n━━━━━━━━━━━━━━━━━\n{icon} اللقب: *{label}*\n"
            f"⚡ XP: *{stats['xp']}*\n🔥 السلسلة: *{stats['streak']}* يوم\n"
            f"🎮 جولات: *{stats['total_games']}*\n")
    if cert:
        text += f"🎖 الشهادة: *{cert}*\n"
    text += "\n📚 *أداء المواد:*\n"
    for key, subj in SUBJECTS.items():
        s = subject_stats.get(key)
        if s and s["games"] > 0:
            acc = int((s["correct"] / (s["games"] * 10)) * 100)
            bar = "▓" * (acc // 10) + "░" * (10 - acc // 10)
            text += f"{subj['icon']} {subj['name']}: `{bar}` {acc}%\n"
        else:
            text += f"{subj['icon']} {subj['name']}: لم تلعب\n"
    b = InlineKeyboardBuilder()
    b.button(text="🎯 توقعات درجاتي", callback_data="grade_prediction_menu")
    b.button(text="← رجوع",           callback_data="back_home")
    b.adjust(1)
    await callback.message.edit_text(text, parse_mode="Markdown", reply_markup=b.
    await callback.answer()

@dp.callback_query(F.data == "weekly_report")
async def weekly_report(callback: CallbackQuery):
    report = db.get_weekly_report(callback.from_user.id)
    acc = int((report["correct"] / (report["games"] * 10)) * 100) if report["game
    grade = ("🌟 ممتاز" if acc>=90 else "⭐ جيد جداً" if acc>=70 else
             "👍 جيد" if acc>=50 else "📚 تحتاج مراجعة" if report["games"] > 0 el
    await callback.message.edit_text(
        f"📈 *تقرير الأسبوعي*\n━━━━━━━━━━━━━━━━━\n"
        f"🎮 جولات: *{report['games']}*\n✅ صحيح: *{report['correct']}*\n"
        f"📊 دقة: *{acc}%*\n⚡ XP: *{report['xp']}*\n\n التقييم: *{grade}*",
        parse_mode="Markdown", reply_markup=back_home_kb())
    await callback.answer()

@dp.callback_query(F.data.startswith("review:"))
async def review_answers(callback: CallbackQuery, state: FSMContext):
```

```
    parts = callback.data.split(":")
    idx, key = int(parts[1]), parts[2]
    data = await state.get_data()
    answers = data.get("answers", [])
    if not answers or idx >= len(answers):
        await callback.answer("انتهت المراجعة ✅", show_alert=True)
        return
    a = answers[idx]
    labels = ["د" ,"ج" ,"ب" ,"أ"]
    opts = ""
    for i, opt in enumerate(a["options"]):
        if i == a["correct"] and i == a["chosen"]: opts += f"✅ {labels[i]}) {opt
        elif i == a["correct"]: opts += f"✅ {labels[i]}) {opt} ← الإجابة الصحيحة\
        elif i == a["chosen"]:  opts += f"❌ {labels[i]}) {opt} ← إجابتك\n"
        else: opts += f"⬜ {labels[i]}) {opt}\n"
    status = "✅ صحيحة" if a["chosen"]==a["correct"] else ("⏰ انتهى الوقت" if a|
    b = InlineKeyboardBuilder()
    if idx > 0: b.button(text="→ السابق", callback_data=f"review:{idx-1}:{key}")
    if idx < len(answers)-1: b.button(text="التالي ←", callback_data=f"review:{id
    b.button(text="← رجوع", callback_data="back_home")
    b.adjust(2, 1)
    await callback.message.edit_text(
        f"📋 *مراجعة {idx+1}/{len(answers)}*\n\n❓ {a['q']}\n\n{opts}\nالنتيجة: {
        parse_mode="Markdown", reply_markup=b.as_markup())
    await callback.answer()

@dp.callback_query(F.data == "reminder_menu")
async def reminder_menu(callback: CallbackQuery):
    stats = db.get_user_stats(callback.from_user.id)
    status = "✅ مفعّل" if stats.get("reminders_on") else "❌ معطّل"
    b = InlineKeyboardBuilder()
    if stats.get("reminders_on"):
        b.button(text="🔕 إيقاف التذكير", callback_data="reminder_off")
    else:
        b.button(text="🔔 8 مساءً",  callback_data="reminder_on:20:00")
        b.button(text="🔔 10 مساءً", callback_data="reminder_on:22:00")
        b.button(text="🔔 7 صباحاً", callback_data="reminder_on:07:00")
    b.button(text="← رجوع", callback_data="back_home")
    b.adjust(1)
    await callback.message.edit_text(f"📅 *التذكير اليومي*\n\nالحالة: {status}",
                                     parse_mode="Markdown", reply_markup=b.as_mar
    await callback.answer()

@dp.callback_query(F.data.startswith("reminder_on:"))
async def reminder_on(callback: CallbackQuery):
    parts = callback.data.split(":")
    time_str = f"{parts[1]}:{parts[2]}"
```

```
    db.set_reminder(callback.from_user.id, True, time_str)
    await callback.message.edit_text(f"✅ *تم تفعيل التذكير الساعة {time_str}* 📚
                                     parse_mode="Markdown", reply_markup=back_hom
    await callback.answer("✅ تم!")

@dp.callback_query(F.data == "reminder_off")
async def reminder_off(callback: CallbackQuery):
    db.set_reminder(callback.from_user.id, False)
    await callback.message.edit_text("🔕 *تم إيقاف التذكير*", parse_mode="Markdow
    await callback.answer()


@dp.callback_query(F.data == "settings")
async def settings_menu(callback: CallbackQuery):
    b = InlineKeyboardBuilder()
    b.button(text="📢 قناة المثابر",      url="https://t.me/AlMuthabir")
    b.button(text="💡 للاقتراح والتواصل", url="https://t.me/AlmuthabirBot")
    b.button(text="← رجوع",               callback_data="back_home")
    b.adjust(1)
    await callback.message.edit_text(
        "⚙️ *الإعدادات*\n"
        "━━━━━━━━━━━━━━━\n\n"
        "📖 *فكرة البوت:*\n"
        "_(قريباً)_\n\n"
        "📢 *قناة البوت:*\n"
        "[t.me/AlMuthabir](https://t.me/AlMuthabir)\n\n"
        "💡 *للاقتراح والتواصل:*\n"
        "@AlmuthabirBot",
        parse_mode="Markdown",
        reply_markup=b.as_markup()
    )
    await callback.answer()

async def send_daily_reminders():
    while True:
        now = datetime.now().strftime("%H:%M")
        for u in db.get_reminder_users():
            if u["reminder_time"] == now:
                try:
                    await bot.send_message(u["user_id"],
                        f"📚 *تذكير يومي!*\n\n يا {u['name']}! لا تنسَ مراجعتك 💪\n
                        parse_mode="Markdown", reply_markup=main_menu_kb())
                except Exception:
                    pass
        await asyncio.sleep(60)

async def main():
```

```
    db.init_db()
    asyncio.create_task(send_daily_reminders())
    await dp.start_polling(bot)

if __name__ == "__main__":
    asyncio.run(main())
```